

PUBLIC KEY ENCRYPTION

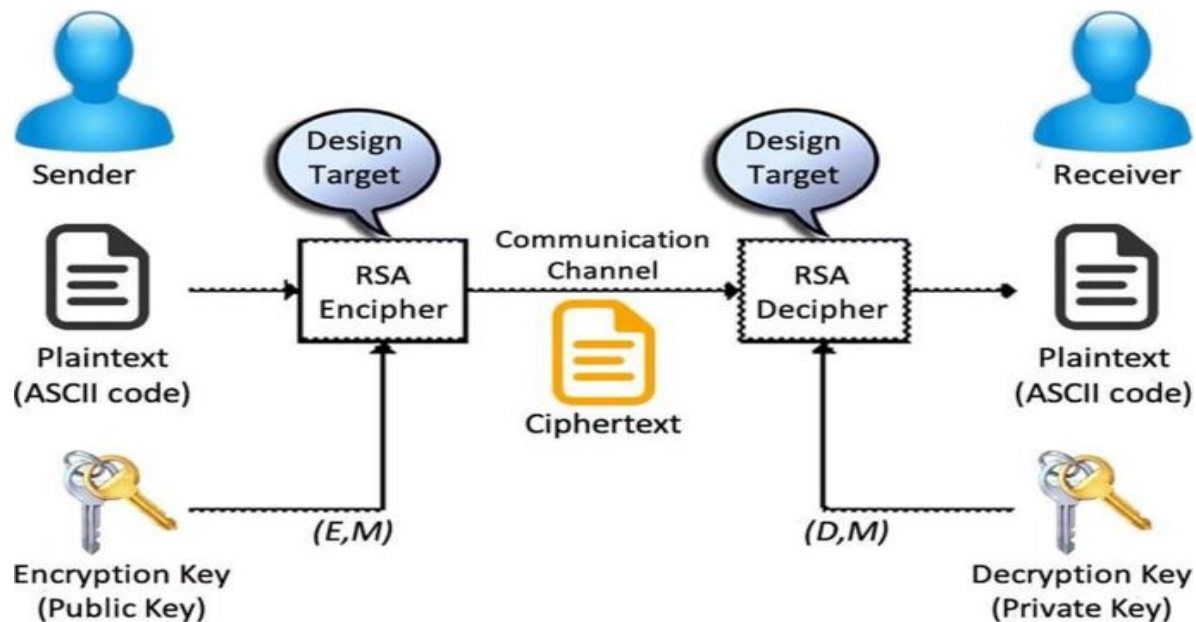
1. RSA Algorithm

1.2 Introduction

- ✚ Invented in 1978 by Ron Rivest, Adi Shamir and Leonard Adleman (RSA)
- ✚ RSA include modern cryptography
- ✚ Security relies on the difficulty of factoring large composite numbers

1.2 How it does work

- ✚ Rivest Shamir Adleman (RSA) is a well-known **public-key or asymmetric cryptographic algorithm**. It protects sensitive data through encryption and decryption using a private and public key pair
- ✚ RSA cipher or RSA algorithm also called **Public Key Encryption**. Asymmetric algorithms are those algorithms in which sender and receiver use different keys for encryption and decryption. Unlike in **Symmetric key** both of encryption and decryption just use one secret key (**shared secret key**)
- ✚ RSA encryption and decryption is based on **number theory** such as *Euclidean Algorithm, Extended Euclidean algorithm, Relative prime, Arithmetic modulo, Inverse modulo, Congruence, prime number, greatest common divisor (GCD), exponential, factoring and Euler's Totient function*. The special property of the number theory is prime number. The prime number are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 and so on. That is, a prime number is a natural number greater than 1 whose only positive factors are itself and 1.
- ✚ Terminology of Asymmetric key Cryptography



Picture. 1: Encryptions and Decryption Process of RSA Algorithm

- ✚ **Public Key** for **Encryption** process on sender side and **Private Key** for **Decryption** process on Receiver side
- ✚ E = Encryption Process, M = Message and D = Decryption Process
- ✚ Decryption cannot be done using a public key. The two keys are linked, but the private key cannot be derived from the public key. The public key is well known, but the private key is secret, and it is known only to the user who owns the key. It means that **everybody** can send a message to the user using user's public key. **But only** the user can decrypt the message using his private key.
- ✚ ASCII TABLE

Dec	Hex	Oct	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr
0	0	000	NULL	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	Start of Header	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	Start of Text	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	End of Text	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	End of Transmission	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	Enquiry	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	Acknowledgment	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	Bell	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	Backspace	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	Horizontal Tab	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	Line feed	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	Vertical Tab	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	Form feed	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	Carriage return	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	Shift Out	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	Shift In	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	Data Link Escape	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	Device Control 1	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	Device Control 2	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	Device Control 3	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	Device Control 4	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	Negative Ack.	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	Synchronous idle	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	End of Trans. Block	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	Cancel	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	End of Medium	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	Substitute	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	Escape	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	File Separator	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	Group Separator	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	Record Separator	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	Unit Separator	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		Del

asciicharstable.com

1.3. RSA Algorithm

1.3.1 Properties / Component / Cryptosystem of RSA

1. P and q prime number
2. $n = p \cdot q$
3. $\varphi(n) = (p - 1)(q - 1)$
4. e = encryption key
 $\gcd(e, \varphi(n)) = 1$
5. d = decryption key
6. m = plaintext
7. c = ciphertext

1.3.2 Generate key pair

1. Select 2 large prime number p dan q, where $p \neq q$.
Typically, each p and q has between **512 and 2048 bits**
2. Compute RSA modulus $n = p \times q$ by multiplying them together
3. Computing Euler's Totient Function: $\varphi(n) = (p - 1)(q - 1)$
4. Choose an integer number **e** as the public key where $1 < e < \varphi(n)$ subject to **e** should be **co-prime** of $\varphi(n)$ so that $\gcd(e, \varphi(n)) = 1$
5. Compute the decryption key (**d**), where $1 < d < \varphi(n)$ subject to $ed \equiv 1 \pmod{\Phi(n)}$ by **congruence** formula:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

$$d = \frac{1 + k \cdot \varphi(n)}{e}, \quad \text{d should be integer using \textbf{Linear Congruence} formula (1)}$$

Where $k = 0, 1, 2, 3, \dots$

OR

$$d = e^{-1} \pmod{\varphi(n)}, \quad \text{find } e^{-1} \text{ using \textbf{Euclidean} and \textbf{Extended Euclidean Algorithm} (2)}$$

OR

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

$$e \cdot d \pmod{\varphi(n)} \equiv 1 \pmod{\varphi(n)}$$

$$d \cdot e \pmod{\varphi(n)} = 1, \quad \text{find d using \textbf{Congruence} formula (3)}$$

6. Calculation by the step 1 until 5 of point **b** above, we get:

- Public key is (**e**, **n**) pair
- Secret Key is (**d**, **n**) pair

7. Encryption and Decryption

a. Encryption

- If the message largest, blocking the plaintext into small blocks such as $m_1, m_2, m_3, \dots$ by the criteria : $0 < m_j < n$, where n is block size.

- Calculate the ciphertext c_i for plaintext m_i using public key (**e**) by the equation:

$$c_i = m_i^e \bmod (n)$$

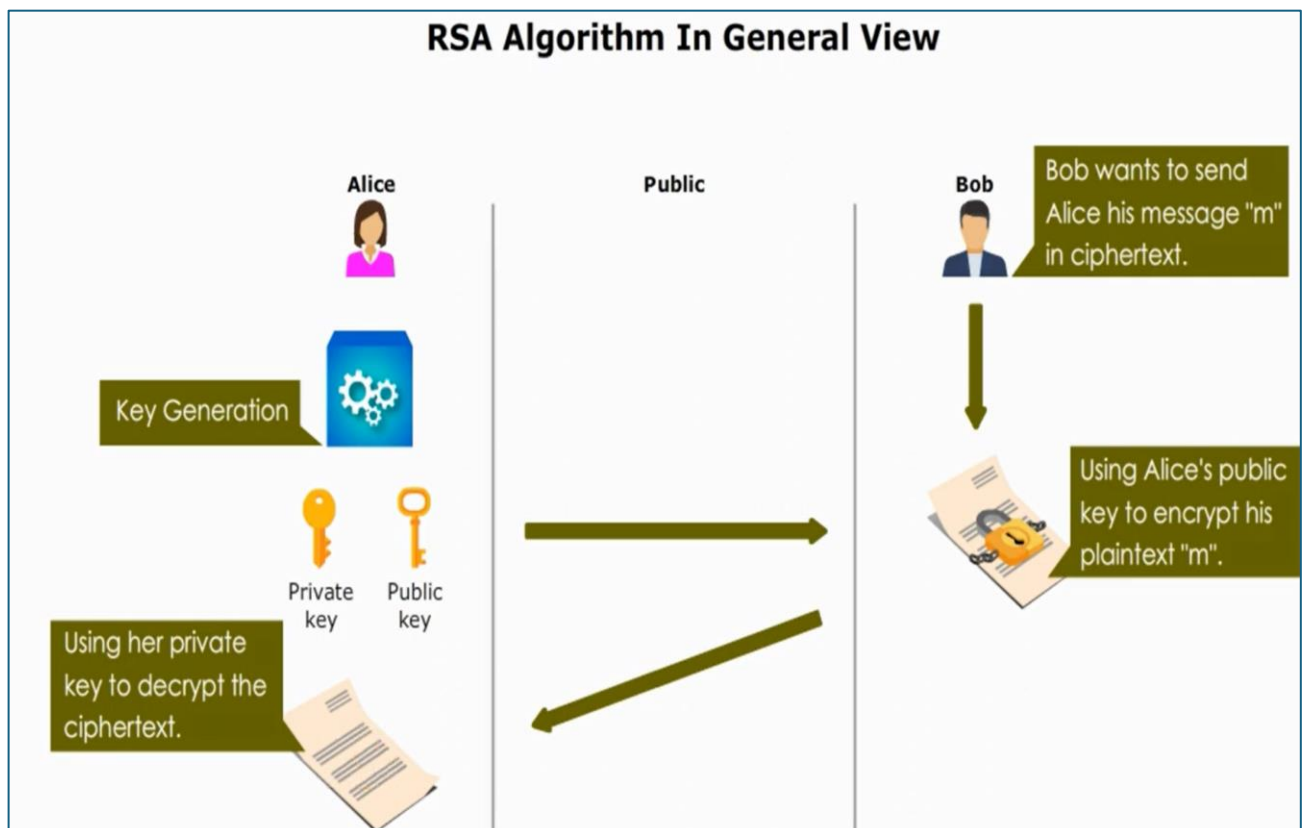
b. Decryption

- Let ciphertext such as $c_1, c_2, c_3, \dots$

- Calculate again the block plaintext m_i of ciphertext c_i using private key (**d**) by the equation:

$$m_i = c_i^d \bmod (n)$$

1.4. RSA Algorithm in General View



- ✚ Suppose that Bob wants to send his message “m” to Alice. “m” is in plaintext to encrypt first
- ✚ Both Alice and Bob agree to use the RSA algorithm and communicate with **insecure channel**
- Our Question are: Who creates keys? Who uses the public key? and Who uses the private key? and how many steps in the wholes process?**
- ✚ Bob must know Alice’s public key to encrypt the message and Alice must use her private key to decrypt the message
- ✚ **The process includes three major steps:**
- ✚ **Step 1:** Alice generates pair of keys: a public key and a private key. She sent the public key to Bob
- ✚ **Step 2:** Bob use Alice’s public key to encrypt his message and sends the encrypted message to Alice
- ✚ **Step 3:** Alice uses her private key to decrypt the encrypted message
- ✚ **Key Generation**

Key Generation (Alice – message receiver)

Select p, q	both p and q are very large prime numbers; $p \neq q$
Calculate $n = p * q$	
Calculate $\Phi(n) = (p-1)(q-1)$	
Select an integer e	• $1 < e < \Phi(n)$ • $\gcd(e, \Phi(n)) = 1$
Public key	$\{e, n\}$
Calculate d	$d * e \bmod \Phi(n) = 1$
Private key	$\{d, n\}$

Encryption (Bob– message sender)

Plaintext m	$m < n$
Ciphertext c	Using Alice’s public key $\{e, n\}$ in the formula: $c = m^e \bmod n$

Key Generation (Alice – message receiver)

Ciphertext c	
Plaintext m	Using her private key $\{d, n\}$ in the formula: $m = c^d \bmod n$

1.5. Manual Calculation RSA

Example: RSA cryptosystem

A RSA cryptosystem uses two prime numbers, 3 and 11. What is the value of ciphertext for a plain text **HERA** using the RSA public-key encryption algorithm?

Explanation:

Step 1: Select two large prime numbers, p , and q .

$$p = 3$$

$$q = 11$$

Step 2: Multiply these numbers to find $n = p \times q$, where n is called the modulus for encryption and decryption.

$$n = p \times q = 3 \times 11$$

$$n = 33$$

Step 3: Calculate Euler Totient function: $\phi(n) = (p - 1) \times (q - 1)$.

$$\phi(n) = (p - 1) \times (q - 1) = \phi(n) = (3 - 1) \times (11 - 1) = \phi(n) = 2 \times 10$$

$$\phi(n) = 20$$

Step 4: Choose a number e less than n , such that n is relatively prime to $(p - 1) \times (q - 1)$.

Choose "e" such that $1 < e < \phi(n)$, e is prime to $\phi(n)$, $\gcd(e, \phi(n)) = 1$.

Let us now choose relative prime e of 20 as 7

- such that $1 < 7 < 20$
- 7 is a prime to 20, to find $\gcd(7, 20)$ we use Euclidean algorithm formula: $a = b \times q + r$

$$20 = 7 \times 2 + 6$$

$$7 = 6 \times 1 + 1$$

since $r = 1$, that mean that $\gcd(7, 20) = 1$

Thus, the public key is $(e, n) = (7, 33)$

Step 5: Calculate d as private key by the formula: $e \cdot d \equiv 1 \pmod{\phi(n)}$

- Using **linear congruence** formula: $d = \frac{1 + k \times (\phi(n))}{e}$

Put $k = 0$

$$D = (1 + 0 \times (20)) / 7 = 1/7 \rightarrow \text{(real number)}$$

Put $k = 1$

$$D = (1 + 1 \times (20)) / 7 = 21/7 = 3 \rightarrow \text{integer number}$$

Thus, the private key is $(d,n) = (3, 33)$

- Using **extended Euclidean algorithm** to find multiplicative inverse of 7 mod 20

Do the Euclidean algorithm: $a = b \times q + r$

$$20 = 7 \times 2 + 6$$

$$7 = 6 \times 1 + \mathbf{1}$$

since $r = 1$, that mean that there exists a multiplicative inverse of 7 mod 20

rewrite:

$$6 = 20 - 7 \times 2 \quad (1)$$

$$1 = 7 - 6 \times 1 \quad (2)$$

Do the Extended Euclidean algorithm

Start with last equation (2)

$$1 = 7 - \underline{6} \times 1$$

$$1 = 7 - (20 - 7 \times 2) \times 1 \text{ *substitute equation 1 to 6 number*}$$

$$1 = 7 - 20 \times 1 + 7 \times 2$$

$$1 = 7 \times 1 - 20 \times 1 + 7 \times 2$$

$$1 = 7 \times (\mathbf{3}) - 20 \times 1$$

$$1 = 1$$

Hence, **3** is a multiplicative inverse for 3 mod 20

Thus, the private key is $(d,n) = (\mathbf{3},33)$

- Using **Congruence** formula:

$$d \cdot e \bmod(\varphi(n)) = 1$$

$$(\mathbf{?} \times 7) \bmod 20 = 1$$

$$(\mathbf{3} \times 7) \bmod 20 = 1$$

$$(21) \bmod 20 = 1$$

Hence, **3** is value of **d**

Thus, the private key $(d,n) = (\mathbf{3}, 33)$

Step 6: Based on the calculation from step 1 to step 5, we have:

- Public key $(e,n) = (7, 33)$
- Private key $(d,n) = (3, 33)$

Step 7: Encryption by plaintext message " m_i " using public key (e,n)

- Suppose given the plaintext message "HERA"

Numerical representation of plaintext "HERA" is 07041700, where $m_i < n$

Encryption:

Calculate the ciphertext c_i for plaintext m_i using public key (e,n) by the equation:

$$C_i = m_i^e \bmod (n)$$

$$C_7 = 7^7 \bmod 33 = 823543 \bmod 33 = 28$$

$$C_4 = 4^7 \bmod 33 = 16384 \bmod 33 = 16$$

$$C_{17} = 17^7 \bmod 33 = 410338673 \bmod 33 = 8$$

$$C_0 = 0^7 \bmod 33 = 0 \bmod 33 = 0$$

Thus, the ciphertext is 28 16 08 00

- Decryption:

Calculate again the block plaintext m_i of ciphertext c_i using private key (d,n) by the equation:

$$M_i = c_i^d \bmod (n)$$

$$M_{28} = 28^3 \bmod 33 = 21952 \bmod 33 = 7$$

$$M_{16} = 16^3 \bmod 33 = 4096 \bmod 33 = 4$$

$$M_{08} = 8^3 \bmod 33 = 512 \bmod 33 = 17$$

$$M_{00} = 0^3 \bmod 33 = 0 \bmod 33 = 0$$

Hence, the plaintext is 07 04 17 00 → HERA

Thus, example to encryption and decryption process of RSA algorithm already done...!!

Notes: Another numerical representation of any message in character format is we can use ASCII table to convert it, if the chosen p and q of prime number are greater than block size n

For example:

If $p = 47$, $q = 71$, $e = 79$ and given the plaintext message "HERA" then

$$N = p \times q = 3337$$

$$\phi(n) = 3220$$

We can convert the plaintext 'HERA' → 72 69 82 65 (to decimal number) then do the encryption and decryption process like in the example cryptosystem above.

Exercise: For you students, GUYS

Zeny and Jacky are students on different university and agree to use the RSA algorithm to deliver message over the public insecure channel. Zany wants to send Jacky a message “GUYS, AMI ESTUDANTE ENGINHARIA INFORMATIKA HERA UNTL MAK NE’E”.

Question for you students, guys:

- Illustrate scheme communication between Zeny and Jacky over the public insecure channel
- who’s party to start generating keys?
- Who uses the public?
- Who uses the private key?
- How many steps in the wholes process?

1.6. Implementation RSA in Python Program

Laboratory activity

2. Diffie Helman Key Exchange Algorithm

2.1 Introduction (Indonesian version: [PDF FILE](#))

- ✚ The Diffie–Hellman (DH) Algorithm is a key-exchange protocol that enables two parties communicating over public channel to establish a mutual secret without it being transmitted over the Internet.
- ✚ DH enables the two to use a public key to encrypt and decrypt their conversation or data using symmetric cryptography.
- ✚ DH is one of the first practical implementations of asymmetric encryption or public-key cryptography (PKC).
- ✚ It was published in 1976 by Whitfield Diffie and Martin Hellman

2.2 How Diffie-Helman Work

The Diffie-Hellman key exchange is based on the discrete logarithm problem, which is considered to be computationally hard. In other words, it is difficult to find the secret key by solving the mathematical problem used in the algorithm. The security of DH relies on the difficulty of calculating the private keys a and b from the public keys A and B . Application of Diffie-Hellman is widely used in internet security protocols, such as SSL/TLS, SSH, and VPNs, to establish a secure connection between two parties. DH is

used to establish a shared secret key for encryption and decryption of the data exchanged between two remote networks.

The DH algorithm works as follows:

1. Consider a prime number p
2. Select g such that $g < p$ and g is primitive root of p

How to test primitive root g modulo p ?

Example 1: Is 2 a primitive root modulo 5?	<i>primitive root</i> <i>g modulo p</i>	Example 2: Is 4 a primitive root modulo 5 ?
$2^1 \bmod 5 = 2$ $2^2 \bmod 5 = 4$ $2^3 \bmod 5 = 3$ $2^4 \bmod 5 = 1$ Based on the criteria 1) and 2) Example 1 is YES	$g^1 \bmod p$ $g^2 \bmod p$ $g^3 \bmod p$ $\dots$ $\dots$ $g^{p-1} \bmod p$ 1). $1 < q < p$ 2). The modular results must be distinct	$4^1 \bmod 5 = 4$ $4^2 \bmod 5 = 1$ $4^3 \bmod 5 = 4$ $4^4 \bmod 5 = 1$ Based on the criteria 1) and 2) Example 2 is NO

3. Assume private key X_A as sender and $X_A < p$

Calculate $Y_A = (g)^{X_A} \bmod p$

Where: $X_A = \text{Private key of } A$ and $Y_A = \text{Public key of } A$

So, the key pair is (X_A, Y_A)

4. Assume private key X_B as receiver and $X_B < p$

Calculate $Y_B = (g)^{X_B} \bmod p$,

Where: $X_B = \text{Private key of } B$ and $Y_B = \text{Public key of } B$

So, the key pair is (X_B, Y_B)

5. Calculate secret keys k_1 and k_2 (for exchanging)

$K_1 \rightarrow \text{Person 1}$ and $k_2 \rightarrow \text{Person 2}$

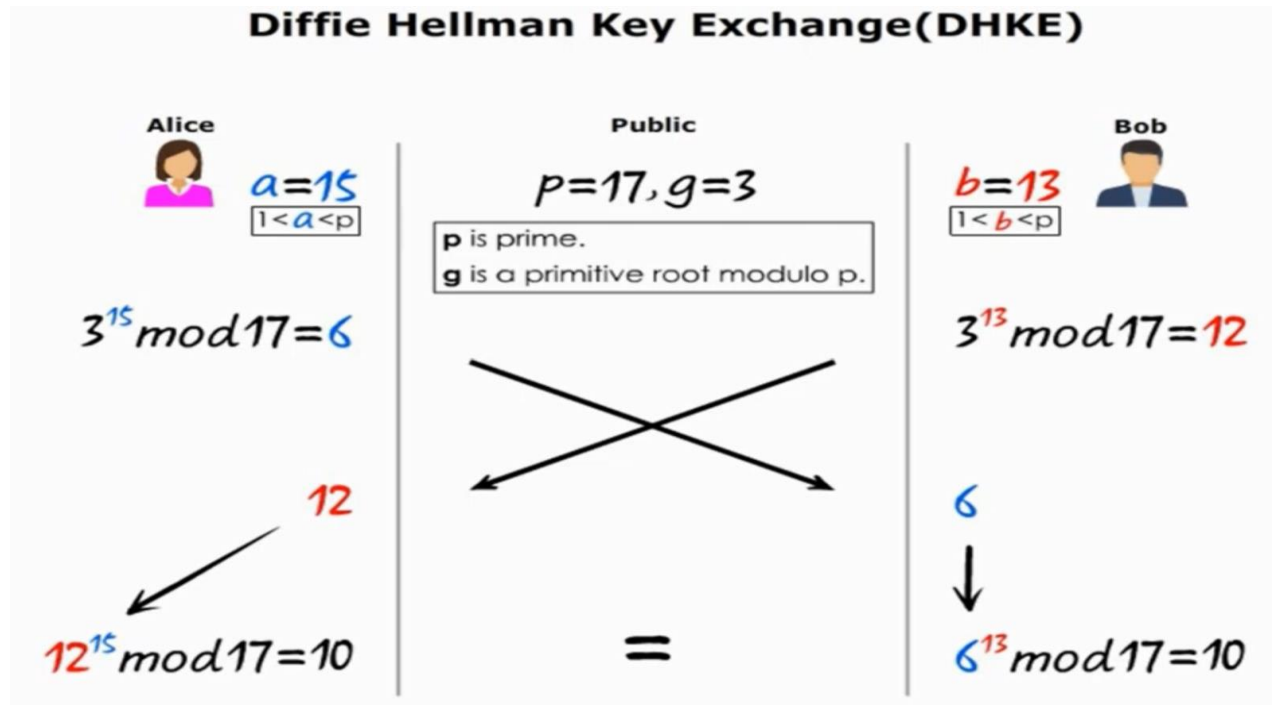
$$K_1 = (Y_B)^{X_A} \bmod p$$

$$K_2 = (Y_A)^{X_B} \bmod p$$

6. After calculating K_1 and K_2

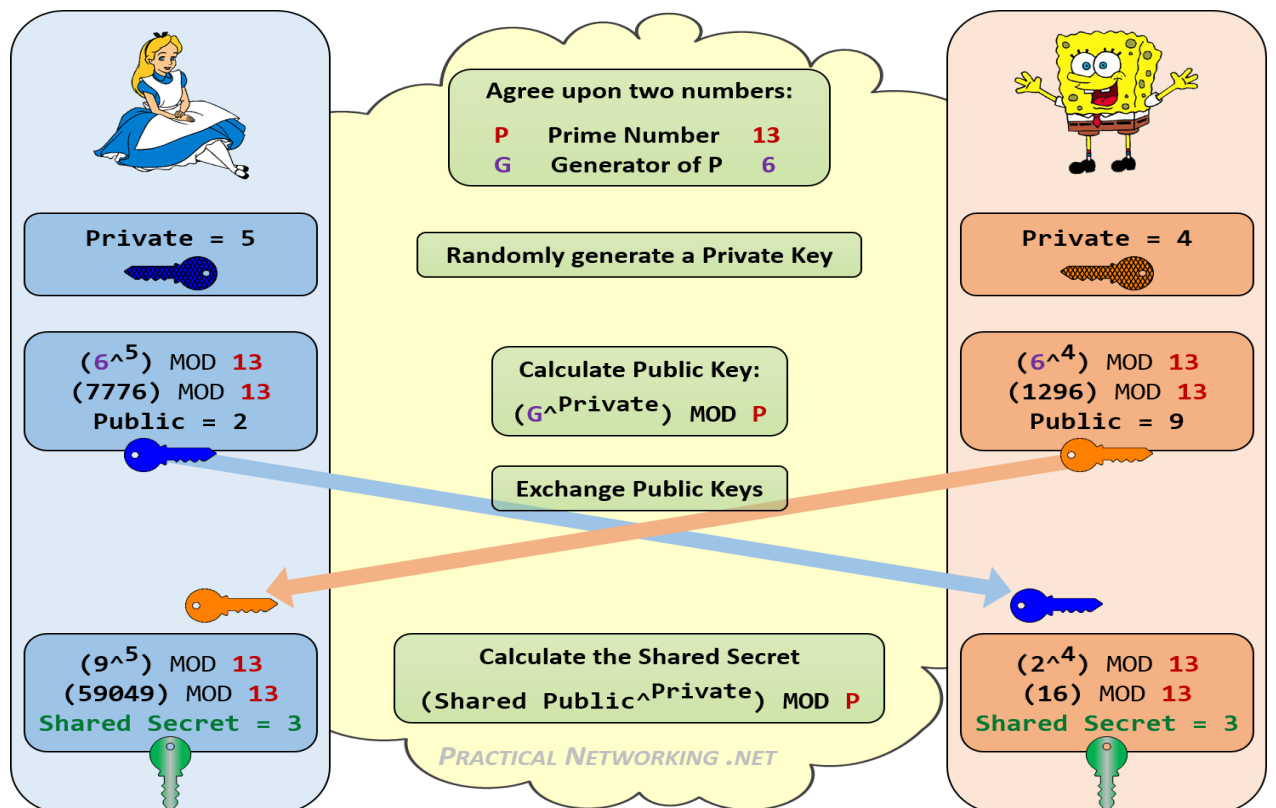
If $K_1 = K_2$, then SUCCESS

Example 1 : DHKE Protocol



Thus, $K1 = K2 \rightarrow$ Success

Example 2 : DHKE Protocol



Thus, $K1 = K2 \rightarrow$ Success

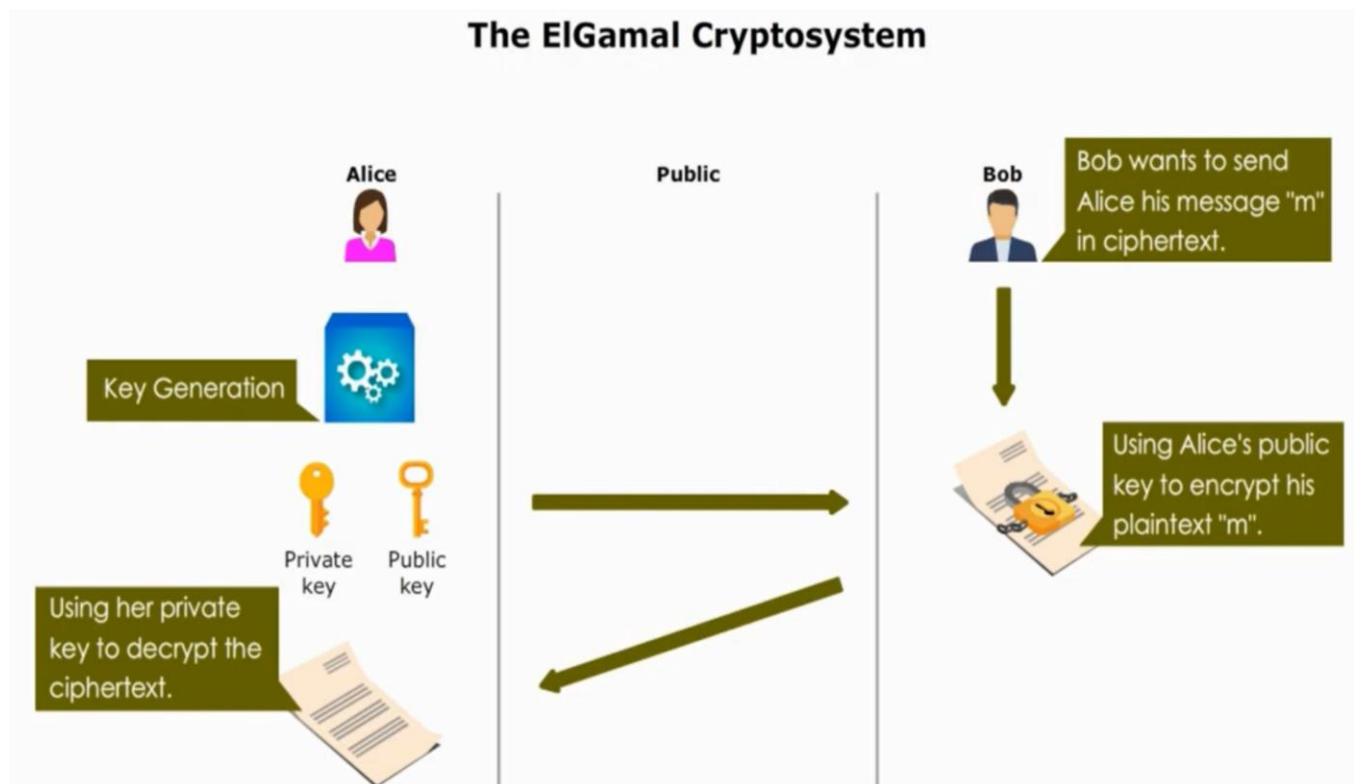
3. ElGamal Algorithm (indonesian [PDF FILE](#), [PPTX FILE](#))

- ✚ The ElGamal encryption algorithm is a public key encryption scheme that consists of three parts: Key generations, encryption and decryption.
- ✚ An asymmetric key encryption algorithm (public key and private key) based on the Diffie-Hellman Key Exchange (DHKE)
- ✚ Created in 1985 by Taher Elgamal, an Egyptian Cryptographer and Entrepreneur.
- ✚ He is also a major contributor to digital signature standard (DSS)
- ✚ The first paper publishes on topic "*A public key cryptosystem and a signature scheme based on discrete logarithms*"
- ✚ The security of this algorithm relies on the difficulty of calculating the **DISCRET LOGARITHMICS**

3.1 How it does work

Elgamal algorithm is a public key cryptosystem. It uses asymmetric key encryption for communicating between two parties and encrypting the message. So, when two user A and user B want to communicate to deliver message with insecure channel, they are using the public key for encryption and private key for decryption.

Here is scenario of the Elgamal algorithm:



The Elgamal algorithm works in three major steps:

a. Key generation procedure – **Alice as message RECEIVER**

1. Select a large prime number p
2. Find a primitive root g of $p, g < p$

Example 1: Is 2 a primitive root modulo 5?	<i>primitive root</i> <i>g modulo p</i>	Example 2: Is 4 a primitive root modulo 5 ?
$2^1 \bmod 5 = 2$	$g^1 \bmod p$	$4^1 \bmod 5 = 4$
$2^2 \bmod 5 = 4$	$g^2 \bmod p$	$4^2 \bmod 5 = 1$
$2^3 \bmod 5 = 3$	$g^3 \bmod p$	$4^3 \bmod 5 = 4$
$2^4 \bmod 5 = 1$	..	$4^4 \bmod 5 = 1$
Based on the criteria 1) and 2) Example 1 is YES	..	Based on the criteria 1) and 2) Example 2 is NO
	$g^{p-1} \bmod p$	

3. Choose a random integer α as her private key, $2 \leq \alpha \leq p - 2$
4. Compute : $e = g^\alpha \bmod p$
5. Key pairs (public key, private key)
 - Public key (e, g, p)
 - Private key (α, p)

b. Encryption process - **Bob as message SENDER**

1. If the message largest, blocking the plaintext into small blocks such as $m_1, m_2, m_3, \dots$
by the criteria : $0 < m_j < n$, where n is block size.

2. Choose a random number $b, 1 \leq b \leq p - 1$
3. Each block m can be encrypted by formula:

$$C_1 = g^b \bmod p$$

$$C_2 = e^b \bmod p$$

Hence, C_1 and C_2 pairs are ciphertext for message block m .

4. Ciphertext: (C_1, C_2)

Thus, the block size of ciphertext is 2 time of block size of plaintext

c. Decryption process- **Alice as message RECEIVER**

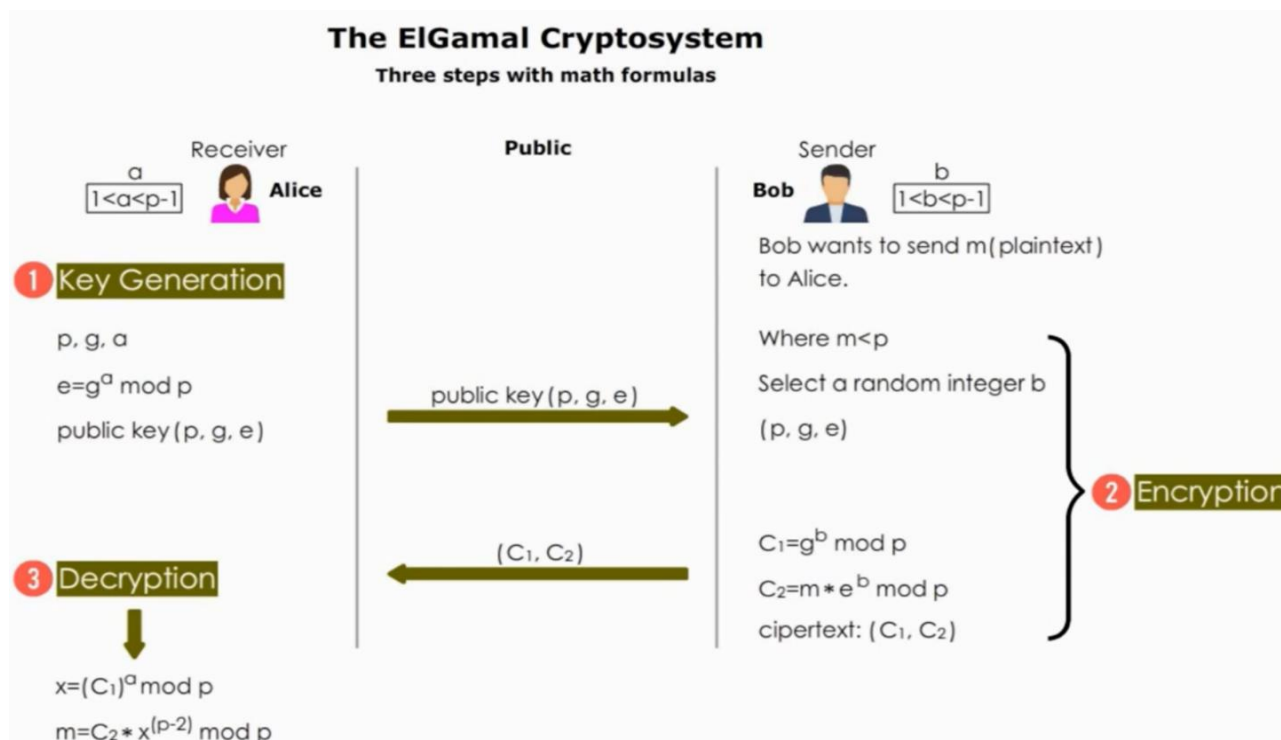
1. Ciphertext: (C_1, C_2)

- Using the private key x to calculate plaintext:

$$x = (C_1)^a \bmod p$$

$$m = C_2 * x^{(p-2)} \bmod p$$

Diagram of three steps of ElGamal Cryptosystem



Example: 1

Bob wants to send message "5" to Alice. Do the encryption and decryption process using Elgamal algorithm.

- Key generation- **Alice as message RECEIVER**

- $P = 11$
- Choose $g = 6$ as primitive root of 11

g primitive root of p ($g \bmod p$)?
$6^1 \bmod 11 = 6 \bmod 11 = 6$
$6^2 \bmod 11 = 36 \bmod 11 = 3$
$6^3 \bmod 11 = 216 \bmod 11 = 7$
$6^4 \bmod 11 = 1296 \bmod 11 = 9$
$6^5 \bmod 11 = 776 \bmod 11 = 10$

$6^6 \bmod 11 = 46656 \bmod 11 = 5$
$6^7 \bmod 11 = 279936 \bmod 11 = 8$
$6^8 \bmod 11 = 1679616 \bmod 11 = 4$
$6^9 \bmod 11 = 10077696 \bmod 11 = 2$
$6^{10} \bmod 11 = 60,466,176 \bmod 11 = 1$

Based on the criteria, all the result should be distinct, so its can be to continue

- **Choose $a = 5$ as private key**
- **Compute: $e = g^a \bmod p = 6^5 \bmod 11 = 776 = 10$**
- **Key Pairs = Public key $(p, g, e = 11, 6, 10)$ and private key $(p, a = 11, 5)$**

2. Encryption- **Bob as message SENDER**

- **Message: $m = 5, m < p$**
- **Select a random integer number $b = 8, 1 < b < p - 1$**
- **Compute two values C1 and C2, where:**

$$C_1 = g^b \bmod p = 6^8 \bmod 11 = 4$$

$$C_2 = m * e^b \bmod p = 5 * 10^8 \bmod 11 = 500000000 \bmod 11 = 5$$
- **So, Ciphertext: $(4, 5)$**

3. Decryption – **Alice as message RECEIVER**

- **Ciphertext: $(C1, C2 = 4, 5)$**
- **Plaintext:**

$$x = (C_1)^a \bmod p = (4)^5 \bmod 11 = 1024 \bmod 11 = 1$$

$$m = C_2 * x^{(p-2)} \bmod p = 5 * 1^{(9)} \bmod 11 = 5 \bmod 11 = 5$$
- **So, plaintext (m) by decryption process is **5****

Exercise:

1. JANY no JACKY sira nain rua kolega estudante iha tinan hirak liu ba. Agora dadaun sira nain rua konkorda atu halao komunikasaun liu husi area publika ne'ebe la ho seguru, utiliza *Elgamal algorithm*. JANNY hakarak manda mensajen ba JACKY **"HAI JACK, HAU KONFUSAUN LOS"**. Perguntas ba guys sira:
 - a. Se mak generate (kria) savi sira (*savi publika no savi privadu*)?
 - b. Se mak utiliza savi publika?
 - c. Se mak utiliza savi privadu?
 - d. Etapa principal hira mak presija iha prosesu tomak atu halao **encryption and decryption**?

3.2. Advantages

1. Security

ElGamal is based on the discrete logarithm problem, which is considered to be a hard problem to solve. This makes it secure against attacks from hackers.

2. Key distribution

The encryption and decryption keys are different, making it easier to distribute keys securely. This allows for secure communication between multiple parties.

3. Digital signatures

ElGamal can also be used for digital signatures, which allows for secure authentication of messages.

3.3. Disadvantages

1. Slow processing

ElGamal is slower compared to other encryption algorithms, especially when used with long keys. This can make it impractical for certain applications that require fast processing speeds.

2. Key size

ElGamal requires larger key sizes to achieve the same level of security as other algorithms. This can make it more difficult to use in some applications.

3. Vulnerability to certain attacks

ElGamal is vulnerable to attacks based on the discrete logarithm problem, such as the index calculus algorithm. This can reduce the security of the algorithm in certain situations.

4. Hash Function

4.1 Introduction

- ✚ Hash function is one technique in cryptographic to secure/guarantee data integrity and message authentication
- ✚ Cryptographic hash functions take data input message and generate a **fixed size result** (message digest/hash code/hash value). It is almost impossible to **regenerate** the input from the result of hash function. One thing to see here is that the hash functions are **not encryption** because it cannot decrypt the input from the output.
- ✚ Generally, hash function called **“one way hashing function”**. So, by definition it should fulfill **two** properties. **One**, it is one way which means one can create a hash value from a message but cannot recreate the message from the hash value. **Two**, it should be **collision** free that is two distinct messages cannot have the same hash value.
- ✚ The hash function formula as follows:

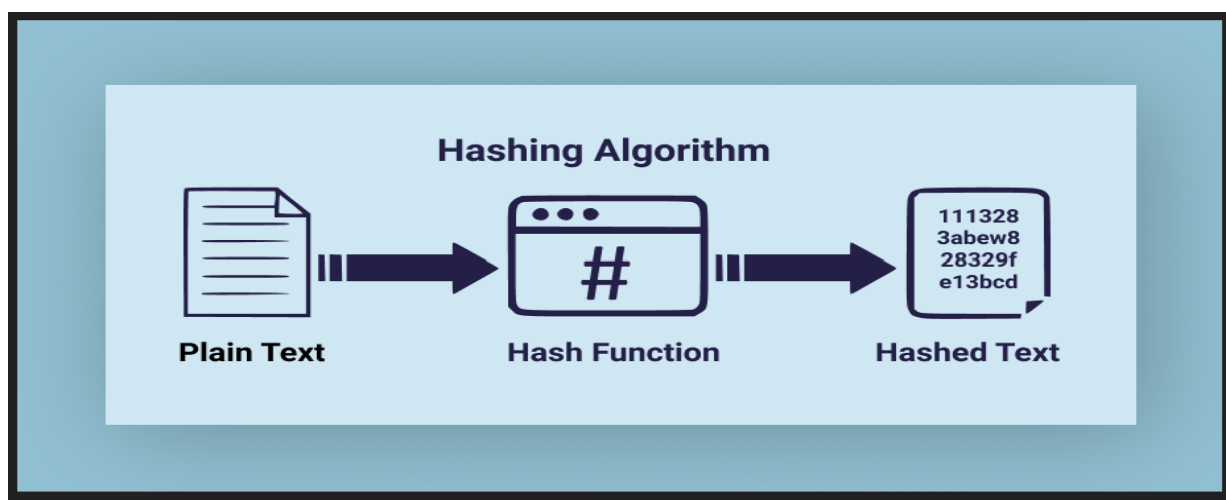
$$h = H(M)$$

Where: h = fixed size result / hash code / message digest

H = generating function

M = input message (variable input – *kualker input size variabel hira deit.....!!*)

$h \ll M$ (size ka medida h kiik liu husi medida M)



Picture 4. a. Hashing function Model

- ✚ Converting Hash Codes to their original value is an impossible task to perform. This is the **main difference between Encryption as Hash Function**.

✚ The other name of hash function:

- Compression function
- Fingerprint
- Cryptographic Checksum
- Message Integrity Check (MIC)
- Manipulation Detection Code (MDC)

4.2 Feature of hash function in Security System

1. One-way function

Hash functions are designed to be one-way functions, meaning that it is easy to compute the hash value for a given input, but difficult to compute the input for a given hash value. This property makes hash functions useful for verifying the integrity of data, as any changes to the data will result in a different hash value.

2. Deterministic

Hash functions are deterministic, meaning that given the same input, the output will always be the same. This makes hash functions useful for verifying the authenticity of data, as any changes to the data will result in a different hash value.

3. Fixed-size output

Hash functions produce a fixed-size output, regardless of the size of the input. This property makes hash functions useful for storing and transmitting data, as the hash value can be stored or transmitted more efficiently than the original data.

4. Collision resistance

Hash functions should be designed to be collision resistant, meaning that it is difficult to find two different inputs that produce the same hash value. This property ensures that attackers cannot create a false message that has the same hash value as a legitimate message.

5. Non-reversible

Hash functions are non-reversible, meaning that it is difficult or impossible to reverse the process of generating a hash value to recover the original input. This property makes hash functions useful for storing passwords or other sensitive information, as the original input cannot be recovered from the hash value.

4.3 Hash Function Algorithm

There are exist several algorithms of one-way hash function in cryptographic shown in the table below:

Table 4.3.1. Types of Hash Algorithm			
Hash Algorithm	Message Digest Size	Block Size	Collision
MD4	128	512	YES
MD5	128	512	YES
SHA-1	160	512	DISABILITY
SHA-256	256	512	NO
SHA-512	512	1024	NO
WHIRLPOOL	512	512	NO
RIPEMD-128	128	512	NO
RIPEMD-160	160	512	NO

1. Message digest 5 (MD5)

1.1. Introduction

- ✚ One of the most widely used Cryptographic hash function is **message digest 5** (MD5)
- ✚ MD5 was designed by Ronald Rivest in 1991 to replace an earlier hash function MD4
- ✚ MD5 creates a 128-bit message digest from the data input which is typically expressed in 32 digits hexadecimal number.
- ✚ MD5 hashes are unique for different inputs regardless of the size of the input.
- ✚ MD5 hashing looks like in this table below (Hashing MD5 nian mak iha tabela kraik ne’e).

Input Message (Variable size length - input)	MD5 Hashing	Message Digest /Hash Value/Hash Code (Fixed-size length - output)
Abc12345		1a64a010767f0725fb52111b0a9e9f84
I am zig-zag 123		59b64ed607da1c93674602f8fb58dc
Ami Estudante Departamentu Engenharia Informatika Hera-UNTL 2024		801f7c475a6e0a6c484afd8f73326113

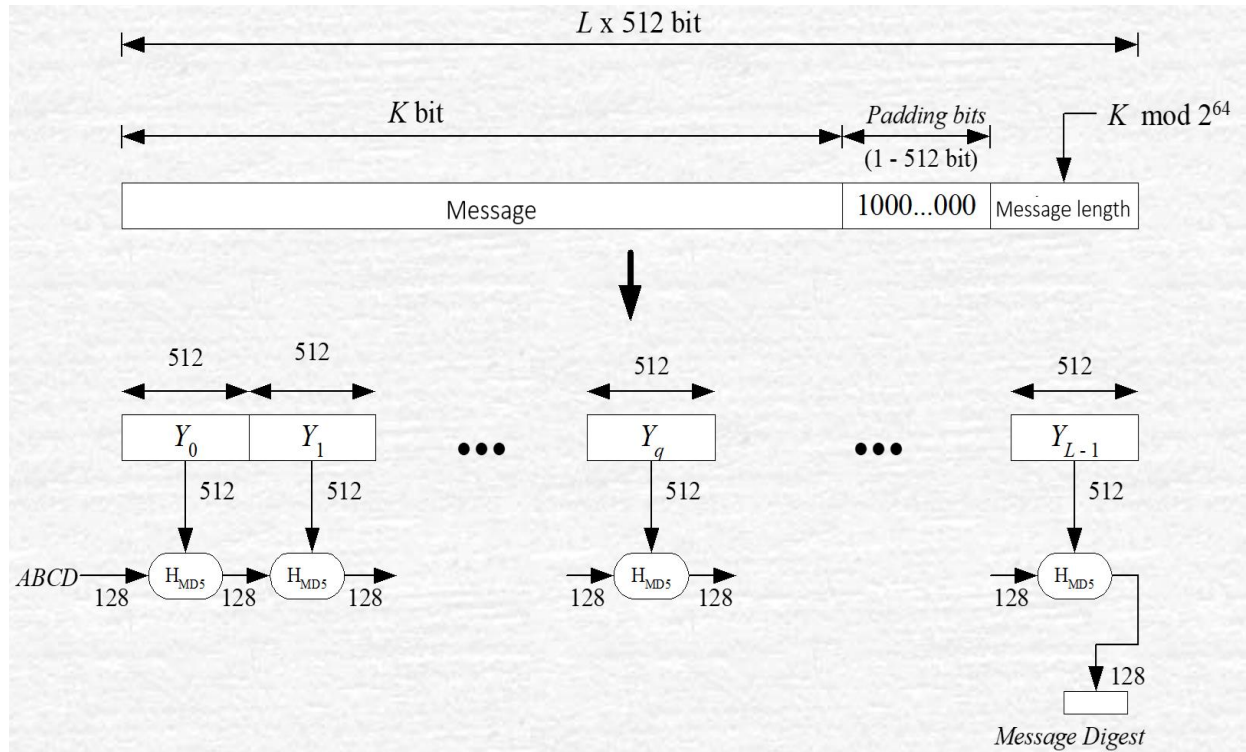
Husi tabela leten hatudu mai katak:

- Badak ka naruk Input message ho variasaun size length (unlimited), maibe message digest ho fixed-sized output (limited, fixu/sertu/so deit 128 bits)

- Generating message ho MD5 Algorithm no rezultadu output mak ho formatu “Hexadecimal” 32 digits

1.2. MD5 Algorithm and How Does it Works?

General view process of MD5 Algorithm



Picture 4.3.1 General view process of MD5 Algorithm

To digest a message, we need to follow 5 steps as follows:

1. Append Padding bits to the message
2. Append Length bits to the message
3. Initialize MD Buffer
4. Process Message in 16-word Block
5. Output

1.3. Example:

2. SHA-1 (Secure Hash Algorithm-1)
3. SHA-256 (Secure Hash Algorithm-256)
4. SHA-512 (Secure Hash Algorithm-512)

4.4 Application of Hash Function in System Security

1. Database indexing: Hashing is used to index and retrieve data efficiently in databases and other data storage systems.
2. Password storage: Hashing is used to store passwords securely by applying a hash function to the password and storing the hashed result, rather than the plain text password.
3. Data compression: Hashing is used in data compression algorithms, such as the Huffman coding algorithm, to encode data efficiently.
4. Search algorithms: Hashing is used to implement search algorithms, such as hash tables and bloom filters, for fast lookups and queries.
5. Cryptography: Hashing is used in cryptography to generate digital signatures, message authentication codes (MACs), and key derivation functions.
6. Load balancing: Hashing is used in load-balancing algorithms, such as consistent hashing, to distribute requests to servers in a network.
7. Blockchain: Hashing is used in blockchain technology, such as the proof-of-work algorithm, to secure the integrity and consensus of the blockchain.
8. Image processing: Hashing is used in image processing applications, such as perceptual hashing, to detect and prevent image duplicates and modifications.
9. File comparison: Hashing is used in file comparison algorithms, such as the MD5 and SHA-1 hash functions, to compare and verify the integrity of files.
10. Fraud detection: Hashing is used in fraud detection and cybersecurity applications, such as intrusion detection and antivirus software, to detect and prevent malicious activities.

4.5 Advantages of Application of Hashing in System Security

1. Efficiency: Hashing allows for fast lookups, searches, and retrievals of data, with an average time complexity of $O(1)$ for hash table lookups.
2. Dynamic: Hashing is a dynamic data structure that can be easily resized, making it suitable for growing and changing datasets.
3. Secure: Hashing provides a secure method for storing and retrieving sensitive information, such as passwords, as the original data is transformed into a hash value that is difficult to reverse.
4. Simple: Hashing is a simple and straightforward concept, making it easy to implement and understand.

5. Scalable: Hashing can be scaled to handle large amounts of data, making it suitable for big data applications.
6. Uniqueness: Hashing ensures the uniqueness of data, as two different inputs will result in two different hash values, avoiding collisions.
7. Verification: Hashing can be used for data verification, such as checking the integrity of files, as even a small change in the input data will result in a different hash value.
8. Space-efficient: Hashing is a space-efficient method for storing and retrieving data, as it only stores the hash values, reducing the amount of memory required.
9. Error detection: Hashing can be used for error detection, as it can detect errors in data transmission, storage, or processing.
10. Speed: Hashing is a fast and efficient method for processing data, making it suitable for real-time and high-performance applications.

4.6 Disadvantage of Application of Hashing in System Security

1. **Collision attacks**

Hash functions are vulnerable to collision attacks, where an attacker tries to find two different inputs that produce the same hash value. This can compromise the security of hash-based protocols, such as digital signatures or message authentication codes.

2. **Rainbow table attacks**

Hash functions are vulnerable to rainbow table attacks, where an attacker precomputes a table of hash values and their corresponding inputs, making it easier to crack password hashes.

3. **Hash function weaknesses**

Some hash functions have known weaknesses, such as the MD5 hash function, which is vulnerable to collision attacks. It is important to choose a hash function that is secure for the intended application.

4. **Limited input size**

Hash functions produce a fixed-size output, regardless of the size of the input. This can lead to collisions if the input size is larger than the hash function output size.

5. Digital Signature Algorithm (DSA)

We can review the goal of cryptography related to the security aspect which was explained in the section before such as:

1. Confidentiality
2. Data Integrity
3. Authentication
4. Non-repudiation

The **aspect (1)** can be resolved by encryption and decryption

- a. Symmetric key Encryption
- b. Asymmetric key (Public key) Encryption

The **aspect (2 to 4)** can be resolved by digital signature with combination public key encryption and hash function. And now, let us give the questions.....!!!!!!

WHAT IS DIGITAL SIGNATURE AND HOW DOES IT WORK?

We have been using **handwritten signatures** to authenticate handwritten or typed messages in the physical world for a long time. Their main use is to bind the signatory with a particular message. **Digital Signature** accomplishes the same thing with digital messages. The signatory, a person or an entity is bound to digital data. This binding is verifiable by the receiver or a third party, if necessary.

Digital Signature in **Cryptography** is a value calculated from the data along with a secret key that only the signer is aware of. In real world, the receiver needs to be assured that the message belongs to the sender and he should not be able to repudiate the origination of that message. This requirement is very crucial in businesses application, since likelihood of a disputes over exchanged data is very high.

There are three algorithms at work in Digital Signatures. They are as follows:

1. Key Generation Algorithms

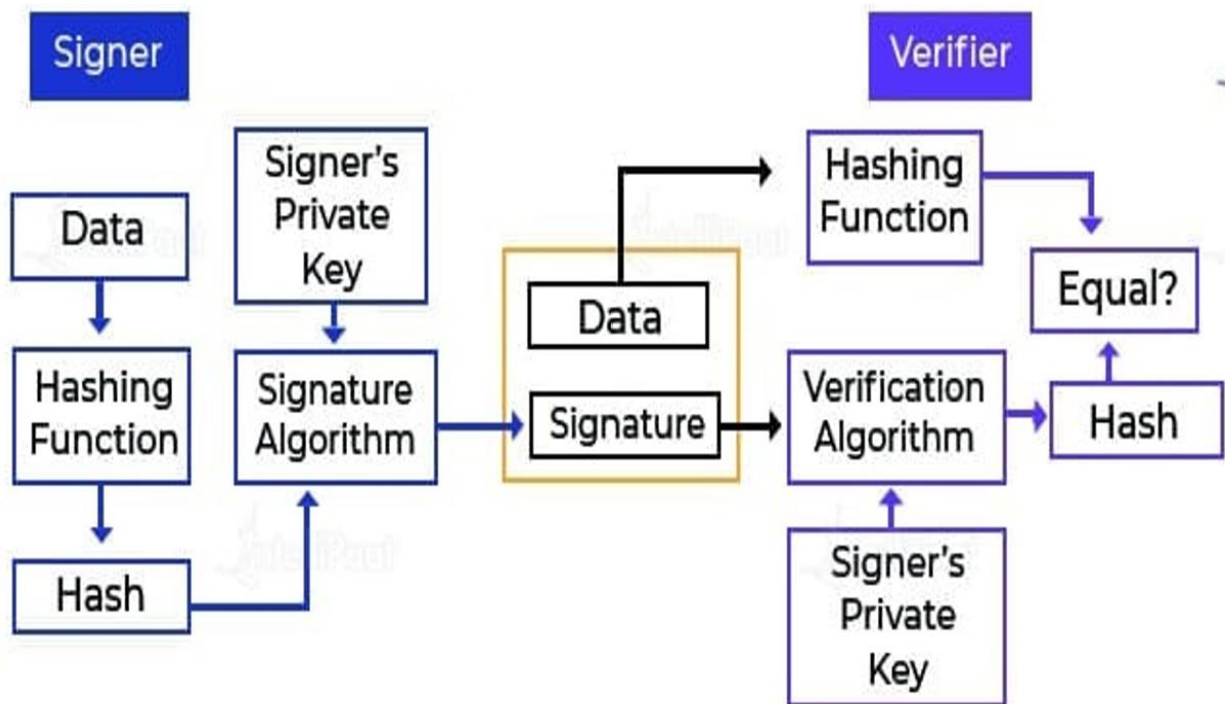
Key Generation Algorithms help ensure authenticity and integrity, or it would be very easy to tamper with the data. They also prevent anyone from pretending to be the sender.

2. Signing Algorithms

Signing Algorithms make one-way hashes of the data that has to be signed. Then they encrypt the hash value using the signature key. The encrypted hash along with the other information is the Digital Signature.

3. Signature Verification Algorithms

Signature Verification Algorithms help process the Digital Signature and the verification key to generate some values. The algorithm also processes the same hash function on the data received and creates a hash value. The general scheme/model of digital signature shown as below:



Picture 1. General Model/Scheme of Digital Signature Algorithm

Explanation of the Scheme:

1. People who adopt the Digital Signature scheme have a public-private key pair.
2. The pairs for encryption/decryption and signing/verifying are usually different. The signature key is the private key that is used for signing and the verification key is the public key.
3. The signer feeds all data to the hashing function and in turn, generates a hash of data.
4. The signature key and hash value are fed into the signature algorithm to help produce a Digital Signature on the given hash. Once the signature is appended to the data, both are sent forward to the verifier.
5. The verifier will then feed the Digital Signature and the verification key into the verification algorithm. The verification algorithm will generate an output value.
6. Verifier is also responsible for running some hashing functions on the data received so it can generate a hash value.

7. Verification is processed by comparing the hash value generated by the verifier and the output of the verification algorithm. The result of this comparison helps the verifier decide if the Digital Signature is valid or not.
8. Nobody else has access to the private key of the signer and the Digital Signature is created using this key, so the signer cannot reject signing the document in the future.

IMPORTANCE OF DIGITAL SIGNATURE

5. Messages Authentication

The private key is only known to the sender. The verifier can use the public key of the sender to validate that the Digital Signature was created by the sender.

6. Data Integrity

If at any time the data is attacked, there will be a discrepancy in the hash value and the verification algorithm as they won't match. Due to this, the receiver will end up rejecting the message and declaring a data breach

7. Non-repudiation

The signer is the only one who is aware of the signature key so, naturally, they are the only ones who can create a specific signature. Whenever there is a dispute, the data along with the Digital Signature can be presented as evidence.

Thus, Privacy, Integrity, Non-Repudiation, and Authentication can be provided as part of a **cryptosystem** if public-key encryption is added to the Digital Signature Scheme

5.1. Digital Signature using RSA Scheme

Algorithm: **Alice sends a message to Bob**

1. Key generation (same as RSA algorithm)

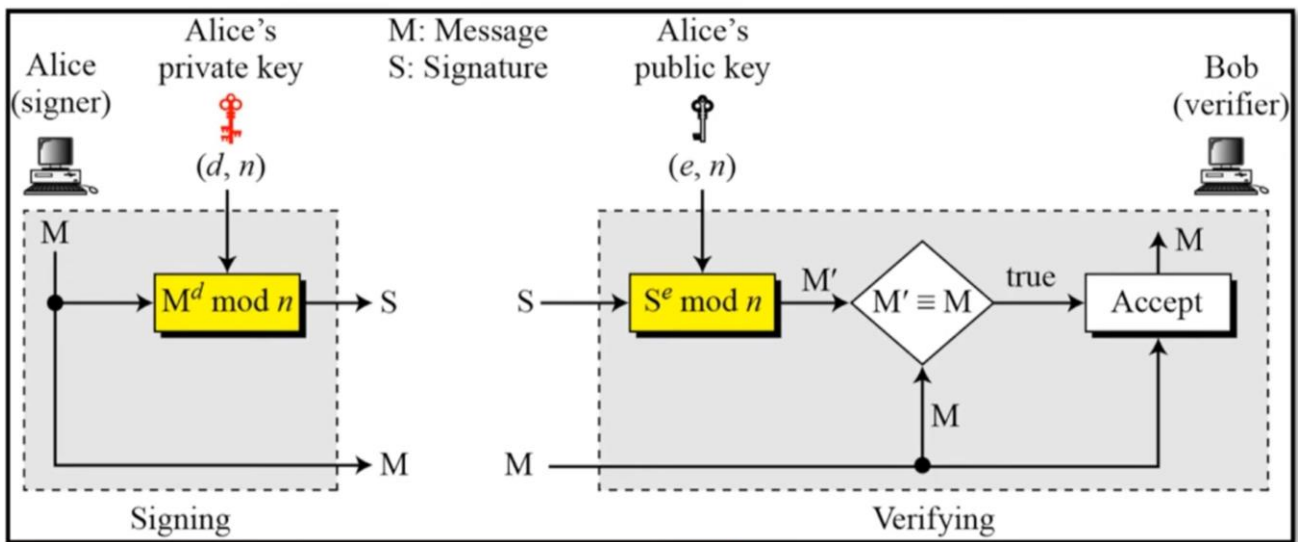
- a. *Alice chooses 2 prime number p and q*
- b. *Calculate $n = p * q$*
- c. *$\varphi(n) = (p - 1) * (q - 1)$*
- d. *She then chooses " e " as the public key, such that $\gcd(e, \varphi(n)) = 1$*
- e. *Calculate " d " as the private key, such that $ed \equiv 1 \pmod{\varphi(n)}$*
- f. *Alice keeps private key " d " and publicly announces e and n*

2. Signing

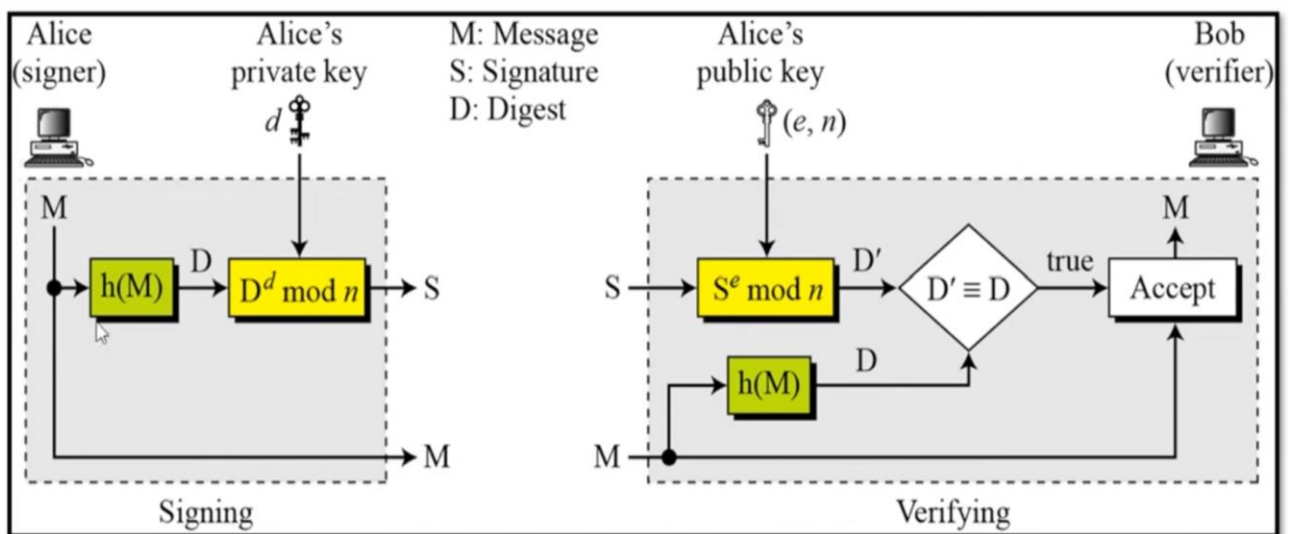
- $S = M^d \bmod p$, uses her private key “d” to create her signature
- Now, this signature “S” and Message “M” is sent to Bob

3. Verifying

- Bob receives “M” and “S”
- Bob applies Alice’s public “e” to the signature to copy of message: $M' = S^e \bmod p$
Bob then compares the values of M and M'
- If both values congruent ($M' \equiv M$), then signature is valid. Its mean that Bob accepts the message



(a) RSA digital signature scheme



(b) RSA digital signature scheme on message digest

Working algorithm of point (b) for Signing and Verifying are the same of point (a).

1. Key generation (same with point “a”)
2. Signing
 - a. Hashing the message “M” into message digest (D). Where: $D = h(M)$
 - b. $S = D^d \bmod p$
 - c. Now, this signature “S” and message “M” is sent to Bob
3. Verifying
 - a. Bob receives “M” and “S”
 - d. Hashing the message “M” into message digest (D). Where: $D = h(M)$
 - b. Bob applies Alice’s public “e” to the signature to copy of message: $D' = S^e \bmod p$
 - c. Bob then compares the values of **D’ and D**
 - d. If both values congruent ($D' \equiv D$), then signature is valid. Its mean that Bob accepts the message

5.2. Digital Signature using ElGamal Scheme

- ✚ The ElGamal Signature Schemes is a digital signature system based on the difficulty level of calculating discrete logarithms and the algebraic properties of modular exponentiation. This is rarely used since it is updated to a better version named Digital Signature Algorithms (DSA).
- ✚ In ElGamal Signature Schemes, the digital signature for the message is created using the sender’s private key and verified using the sender’s public key.
- ✚ The use of discrete logarithms to generate keys is one of the benefits of ElGamal Signature Schemes. When using encryption and decryption methods, a lot of processing ability is used, resulting in encryption that is twice as large as the original size
- ✚ The ElGamal Signature Schemes checks the digital signature’s authenticity and integrity.
- ✚ **Working Algorithm** (Sender=====to=====Receiver)

Calculating at the Sender’s side scheme:

To sign a message **M**, sender first compute the hash, such that hash **$h = H(M)$** is an integer in the range $0 \leq h \leq p - 1$. Sender then form a digital signature as follows:

1. Select a prime number p
2. Find a primitive root g of p, $g < p$

3. Choose a random integer α as her private key, $2 \leq \alpha \leq p - 2$
4. Compute : $e = g^\alpha \text{ mod } p$
5. Generate Key pairs for sender (public key, private key)
 - Public key (e, g, p)
 - Private key (α, p)
6. Generate hash code (h) for the simple text passed
7. $h = H(M)$, {simple text M , is passed through a hash function; hence, we get a hashed code h }
8. Generate random integer k , $\{1 \leq k \leq p - 1\}$ and $\text{gcd}(k, p - 1) = 1$
9. Now calculate the S_1, S_2 , known as signature pair

$$s_1 = g^k \text{ mod } p$$

$$s_2 = k^{-1}(H(M) - \alpha \cdot s_1) \text{ mod } p - 1$$
10. Now we got the signature pair (s_1, s_2)

Calculating at the Receiver's side scheme:

1. Calculating V_1 and V_2
 - $v_1 = g^h \text{ mod } p$
 - $v_2 = e^{s_1} * (s_1)^{s_2} \text{ mod } p$
2. Checking the condition for validation signature
3. if $(v_1 = v_2)$ then signature valid, else signature invalid

Example: Let $p = 19$ and $g = 10$ (on calculating we get $g = 10$)

Calculating at the Sender's side:

1. $P = 19$
2. $g = 10$ (calculating g by primitive root of p)
3. Generating random number $\alpha = 16$
4. Compute : $e = 10^{16} \text{ mod } 19 = 4$
5. Generating key pairs
 - Private key $(\alpha = 16)$
 - Public key $(e, g, p = 4, 10, 19)$
6. Now getting hash code h for the simple text (M)
7. $h = H(M)$, now taking $m = 14$
8. Generate random integer k , such that $\text{gcd}(k, p - 1) = 1$. Now $k = 5$
9. Now calculate the S_1 dan S_2

$$s_1 = g^k \bmod p = 10^5 \bmod 19 = 3$$

$$s_2 = k^{-1}(H(M) - \alpha \cdot s_1) \bmod p - 1 = 5^{-1}(14 - 16 \cdot 3) \bmod 18 = ?$$

Find k^{-1} :

- $k^{-1} \bmod 18$
- What should be multiplied to 5 * ? to get 1 * mod (19)
- $k^{-1} = 11$

$$s_2 = 11(14 - 16 * 3) \bmod 18 = 4$$

10. Thus, the signature pair we get is (3,4)

Calculating at the Receiver's side:

1. Calculating V1 and V2

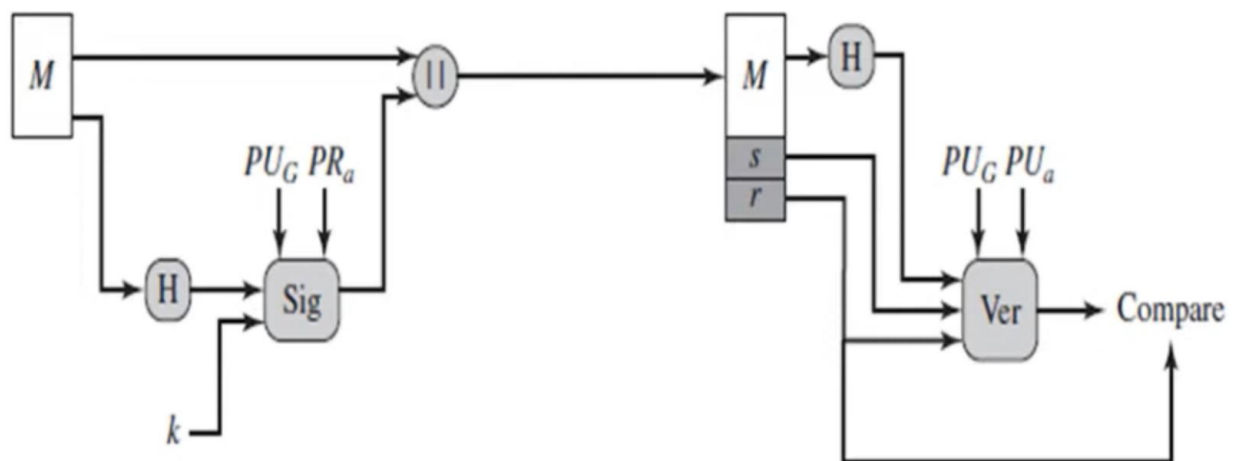
- $v_1 = g^h \bmod p = 10^{14} \bmod 19 = 16$
- $v_2 = e^{s_1} * (s_1)^{s_2} \bmod p = e^{s_1} * (s_1)^{s_2} \bmod p$

2. Checking the condition for signature validation

- Based on the calculation values of V1 = V2, it's mean that a signature is valid
- But, if values V1 and V2 were not equal, then we can understand that in between hackers, any attacker, or any other user might have interpreted in between of which we didn't receive the values to be equal.

5.3. Digital Signature Algorithm (DSA) or Digital Signature Standard (DSS)

🌈 The DSS approach also makes use of a hash function



(a). DSS Scheme

- ✚ The hash code is provided as input to a signature function along with a random number k , generated for this particular signature.
- ✚ The signature function also depends on the sender's private key (PR_a) and a set of parameters known to a group of communicating principle.
- ✚ We can consider this set to constitute a global public key (PU_a)
- ✚ The result is a signature consisting of two components, labeled s and r .
- ✚ At the receiving end, the hash code of the incoming message is generated
- ✚ The signature is input to a verification function
- ✚ The verification function also depends on the global public key as well as the sender's public key (PU_a), which is paired with the sender's private key
- ✚ The signature function is such that only the sender, with knowledge of the private key, could have produced the valid signature.

Working Algorithm (Sender=====to=====Receive):

1. Key generation

Global Public – Key Components (PU_G)

p prime number where $2^{L-1} < p < 2^L$ for $512 \leq L \leq 1024$ and L a multiple of 64
i.e. bit length of between 512 and 1024 in increments of 64 bits

q prime divisor of $(p-1)$, where $2^{159} < q < 2^{160}$, i.e. bits length of 160 bits

$g = h^{(p-1)/q} \bmod p$, where h is any integer with $1 < h < p - 1$ such that
 $h^{(p-1)/q} \bmod p > 1$

User's Private Key

x Random integer or pseudorandom integer with $0 < x < q$

User's Public Key

$$y = g^x \bmod p$$

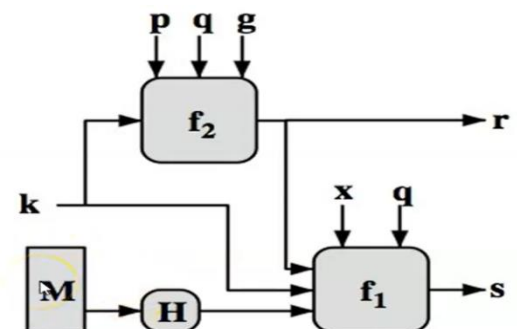
2. Create digital signature (Signing)

User's Per – Message Secret Number

K = number or pseudorandom integer with $0 < k < q$

Signing:

$$r = (g^k \bmod p) \bmod q$$



$$s = [k^{-1}H(M) + xr] \bmod q$$

Signature = (s, r)

3. Signature Verification

Verifying:

$$w = (s')^{-1} \bmod q$$

$$u_1 = [H(M')w] \bmod q$$

$$u_2 = (r')w \bmod q$$

$$v = [(g^{u_1}y^{u_2}) \bmod p] \bmod q$$

Test signature: $v = r$

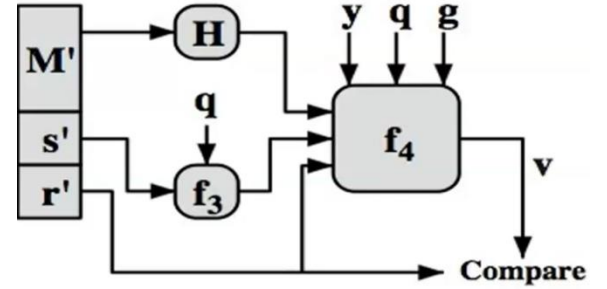
Where:

M = message to be signed

$H(M)$ = hash of M using SHA – 1

M', r', s' = received version of M, r, s

(b). Signing Scheme



$$w = f_3(s', q) = (s')^{-1} \bmod q$$

$$v = f_4(y, q, g, H(M'), w, r')$$

$$= ((g^{H(M')w} \bmod q) y^{r'w} \bmod q) \bmod p \bmod q$$

(c) Verifying Scheme

6. Ds